

Introduction to MySQL

MySQL

It is freely available open source Relational Database Management System (RDBMS).

MySQL was created and is supported by **MySQL, AB**, a company based in Sweden (www.mysql.com). The chief inventor of MySQL was Michael Widenius. MySQL has been named after his daughter **My**.

Features of MySQL

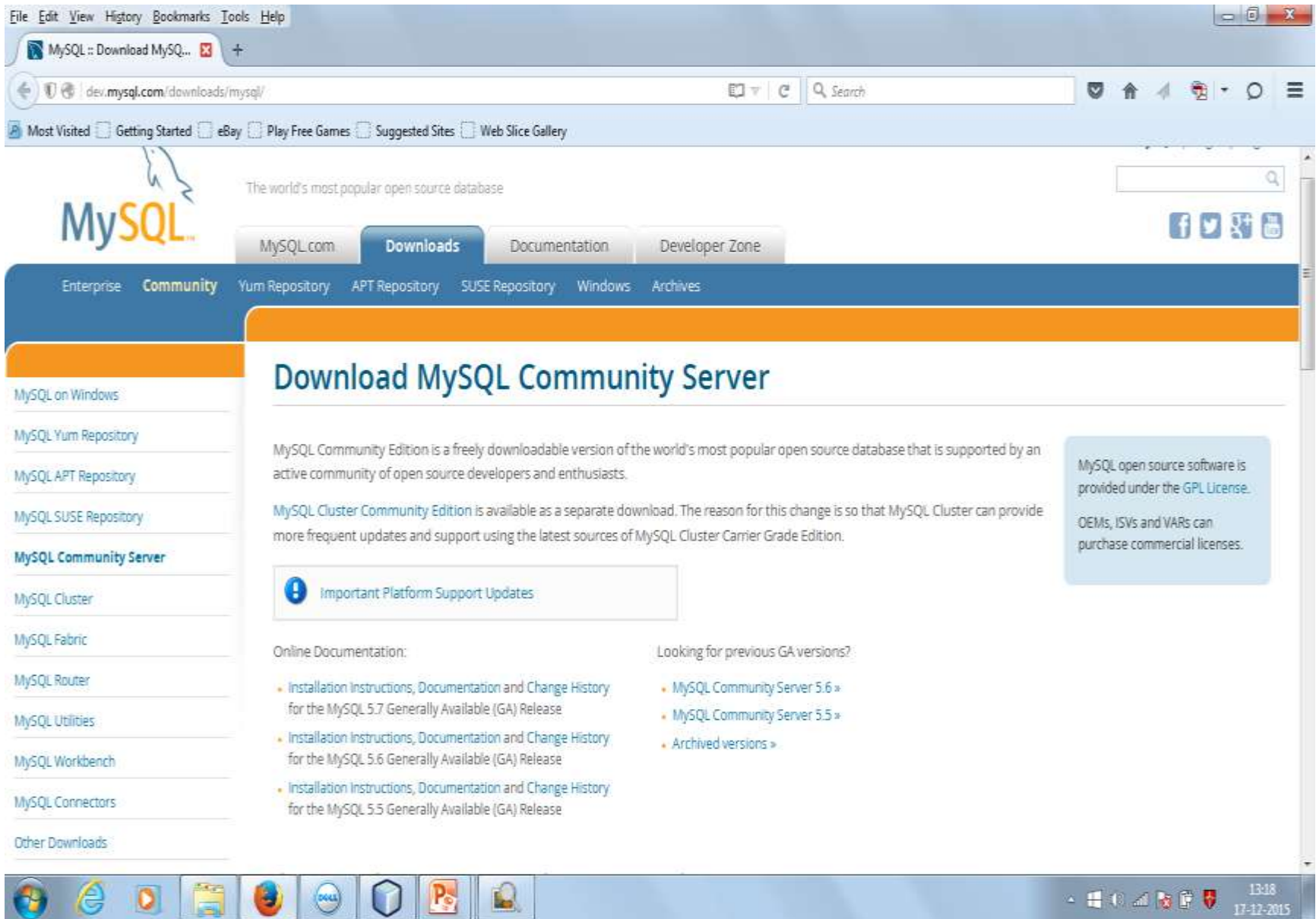
- MySQL is released under an open-source license so it is customizable. It requires no cost or payment for its usage.
- MySQL has superior speed, is easy to use and is reliable.
- MySQL uses a standard form of the well-known ANSI-SQL standards.
- MySQL is a platform independent application which works on many operating systems like Windows, UNIX, LINUX etc. and has compatibility with many languages including JAVA , C++, PHP etc.
- MySQL is an easy to install RDBMS.

DOWNLOADING MySQL

Installation file for MySQL may be downloaded from the link:

<http://dev.mysql.com/downloads/mysql/>

(Choose appropriate download link as per the operating system)



INSTALLING MySQL

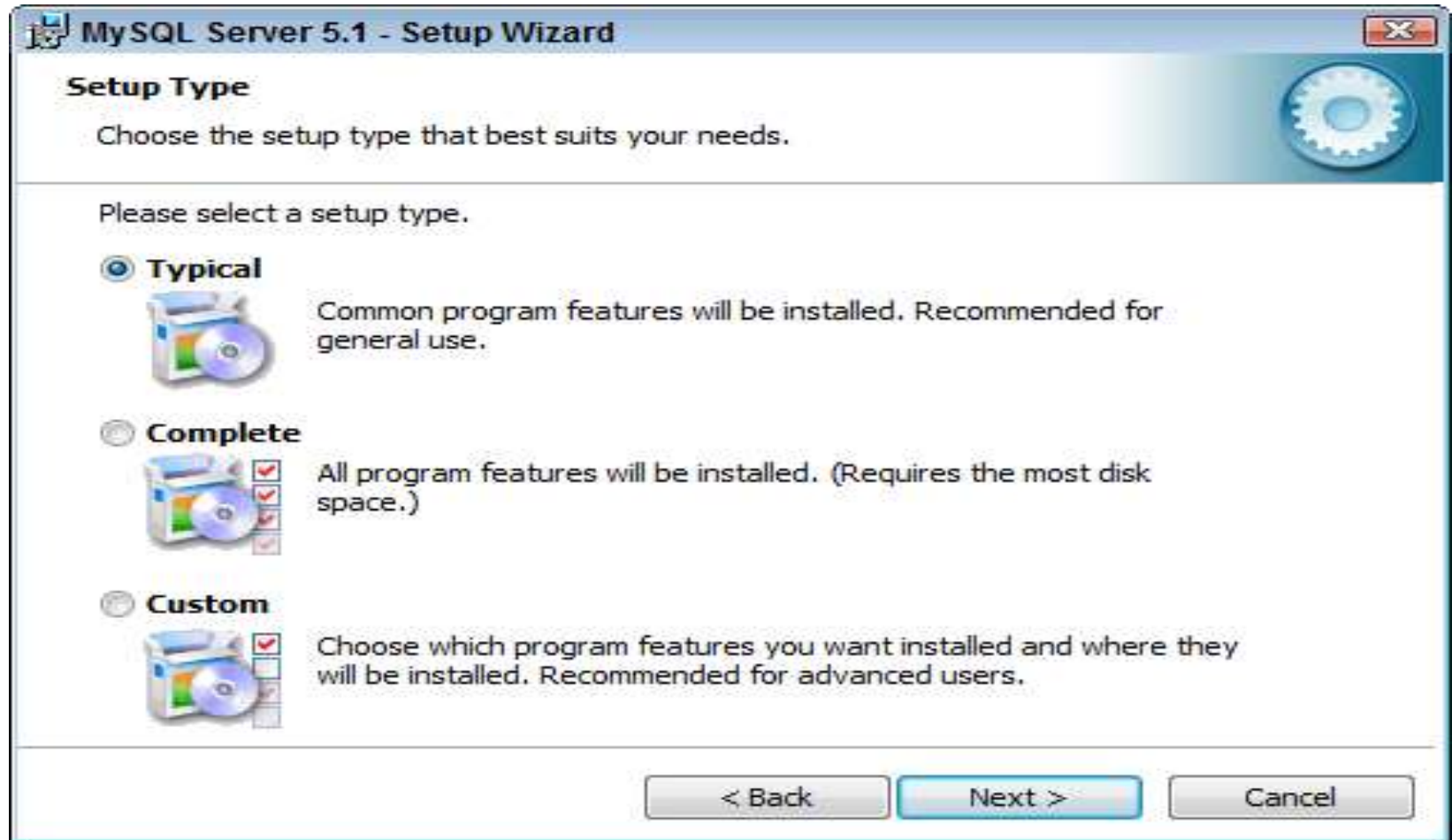


INSTALLING MySQL

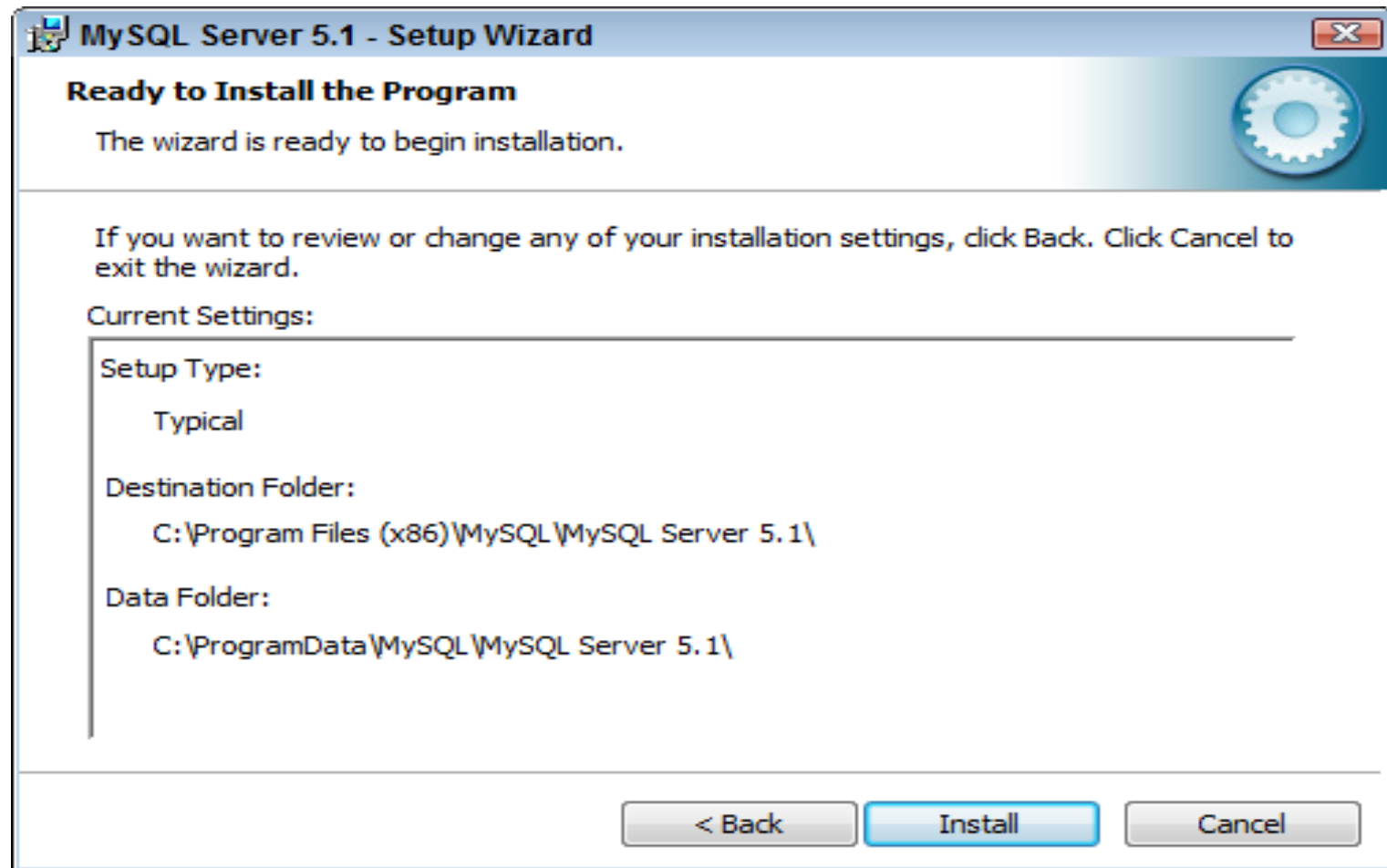
After the installation file has finished downloading, double-click it, which begins the MySQL Setup Wizard.



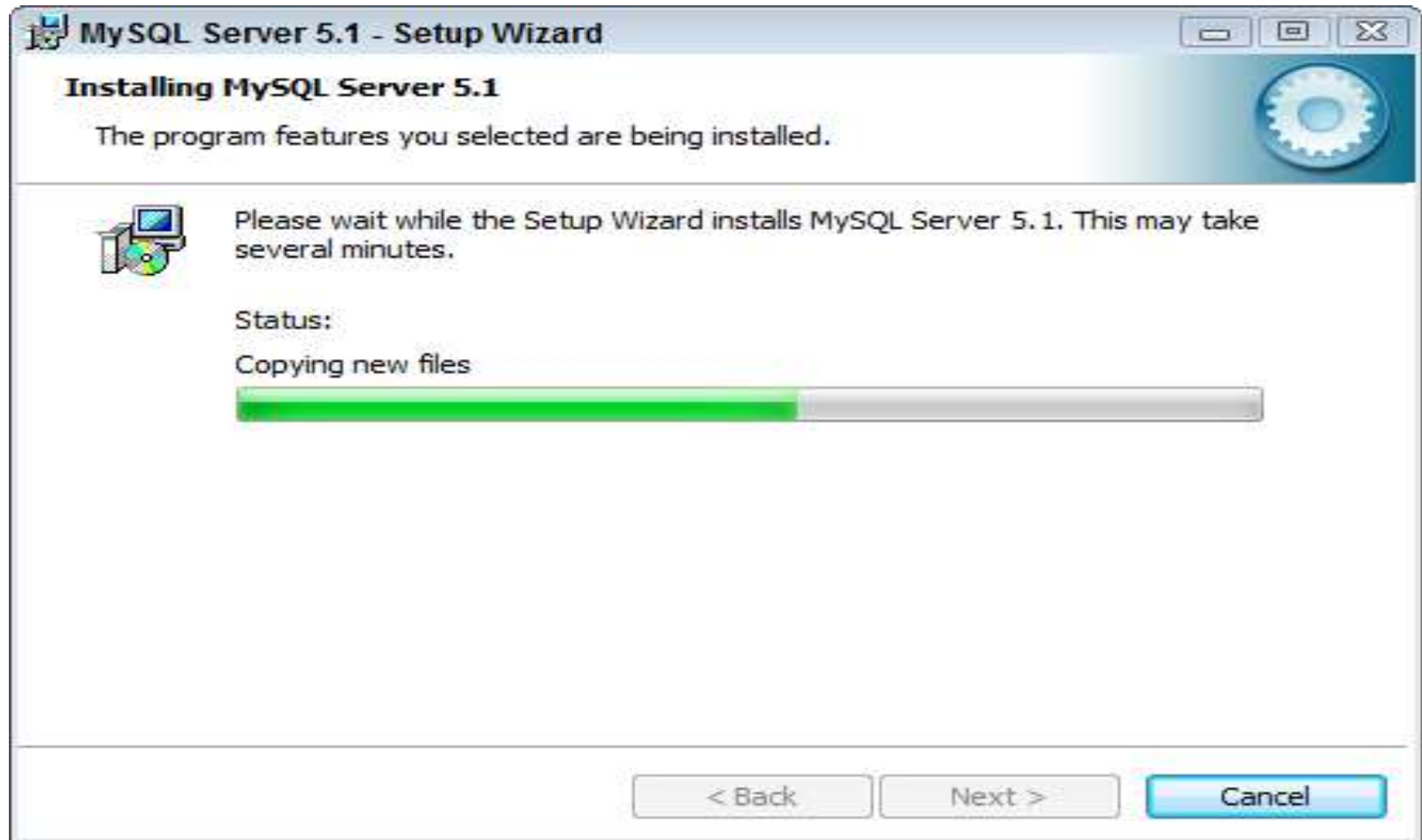




Select the Typical installation option
Click [Next] button



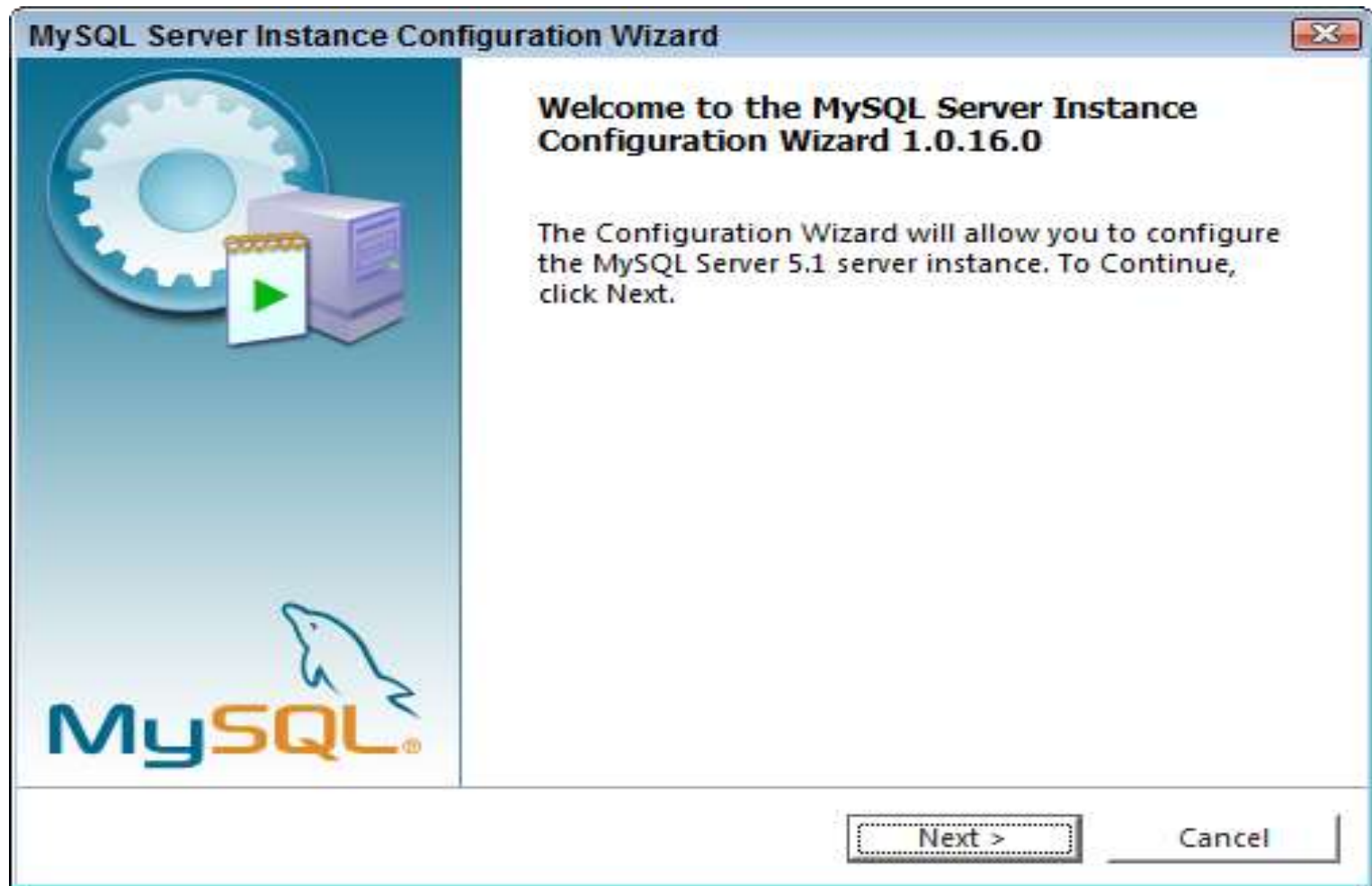
Click [Install] button



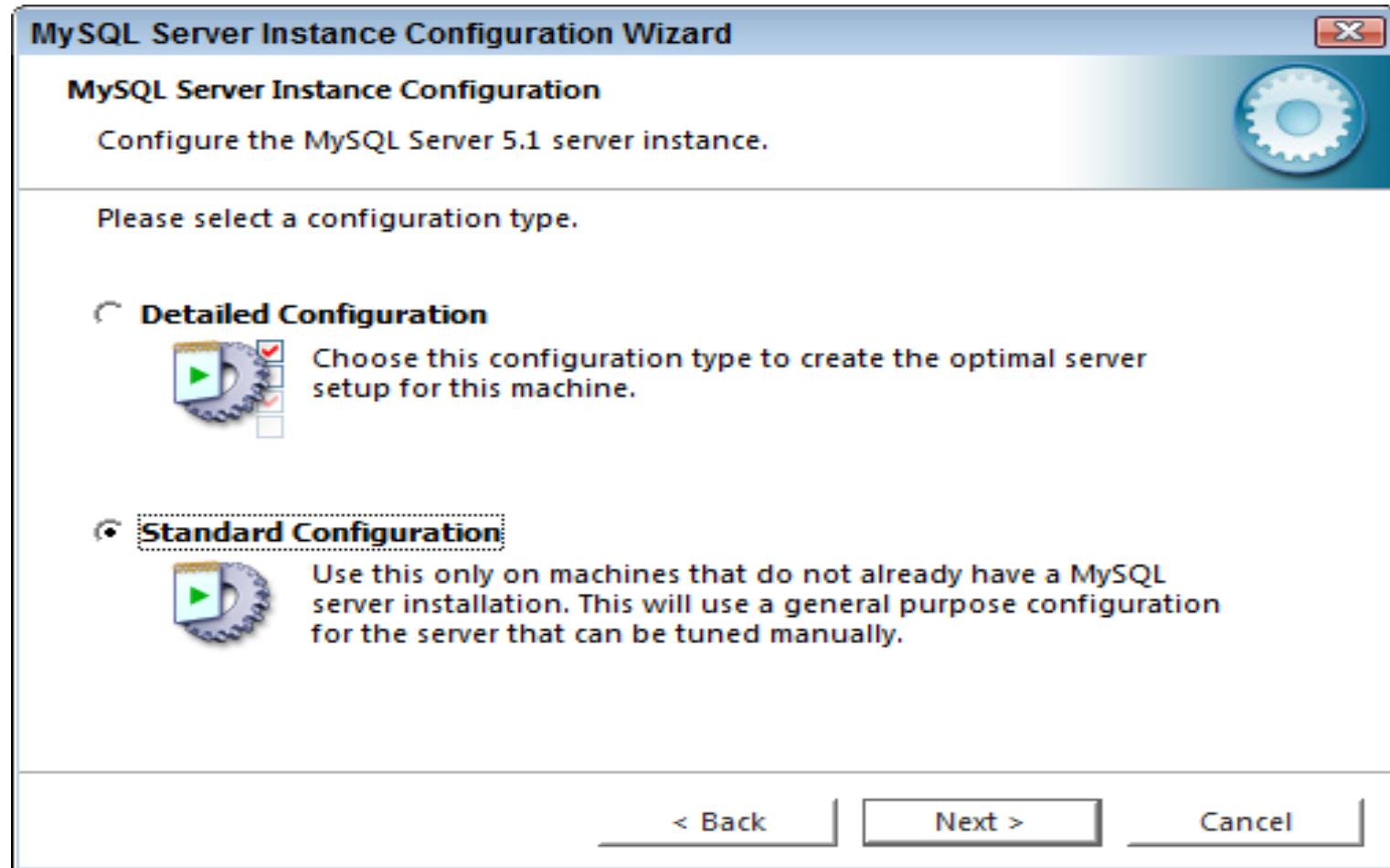
The installation can take some time depending on
your computer's hardware configuration



While configuration could be performed at later time, this tutorial does it as part of the installation process.
Click [Finish] button

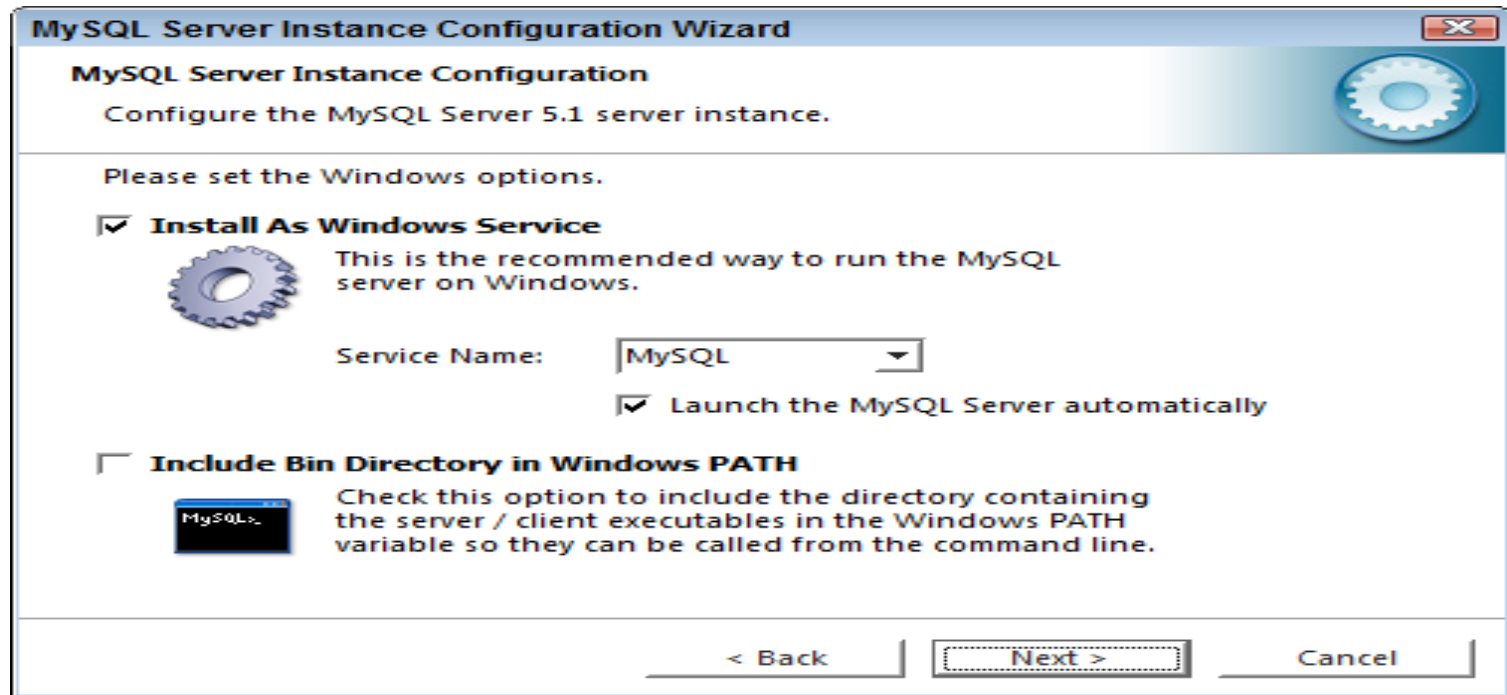


Click [Next] button



Select the Standard Configuration option
Click [Next] button

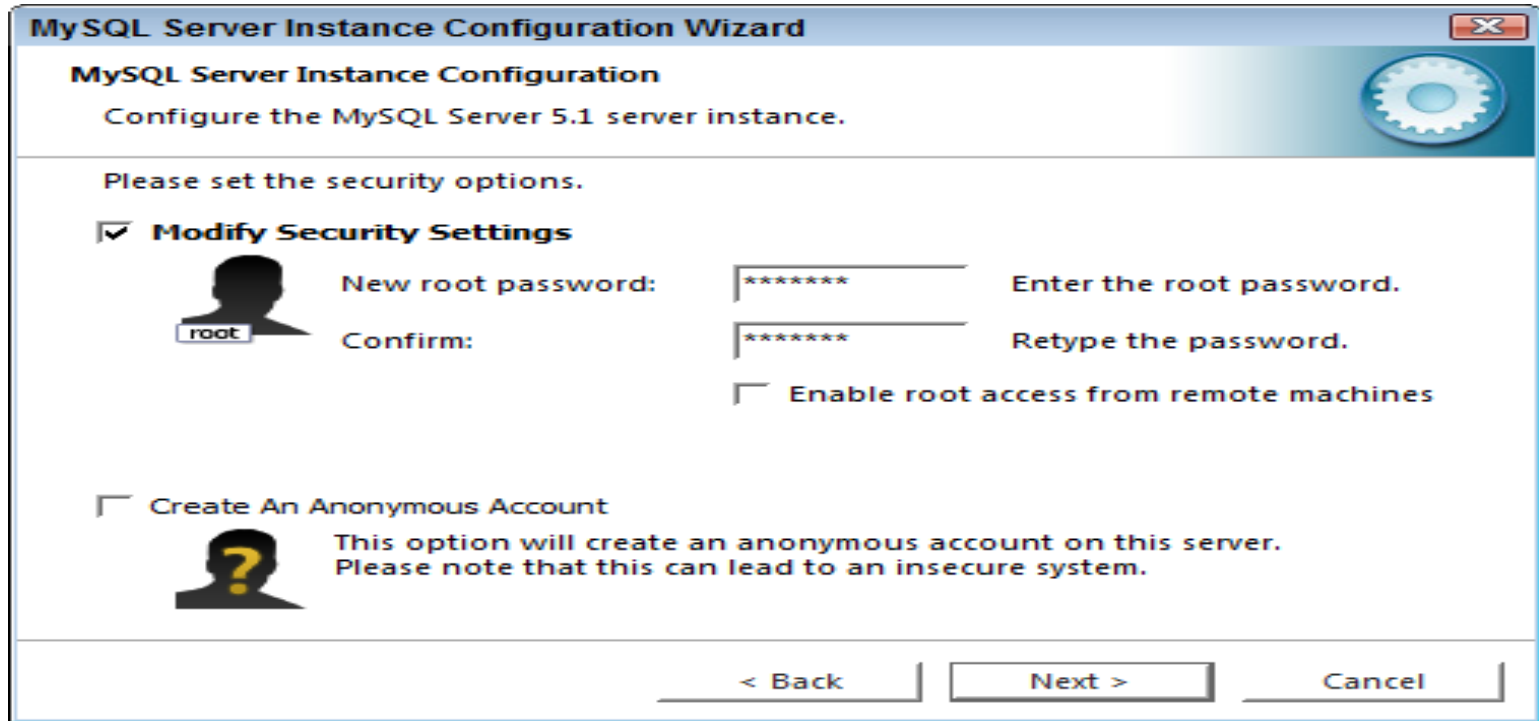
On Windows machines it is recommended to install server-type software as “[Windows Service](#)” as opposed to application. The advantages are that the MySQL server could start automatically on the machine startup, and will run in background thus utilizing fewer resources



Select the Install As Windows Service option; check box to include MySQL \bin directory in [Windows PATH environmental variable](#) – this will facilitate using MySQL from command line.

Click [Next] button

Password for administrative account is the key to the kingdom. Make it reasonably secure and safeguard it. Creating an Anonymous Account would allow anyone to connect to your database server without user ID and password; this could be potentially compromise security in your environment



The image shows a screenshot of the 'MySQL Server Instance Configuration Wizard' window. The title bar reads 'MySQL Server Instance Configuration Wizard'. The main heading is 'MySQL Server Instance Configuration' with a subtitle 'Configure the MySQL Server 5.1 server instance.' and a gear icon. The instruction 'Please set the security options.' is displayed. There are two main sections: 'Modify Security Settings' which is selected with a checked checkbox, and 'Create An Anonymous Account' which is unselected. The 'Modify Security Settings' section includes a user icon labeled 'root', a 'New root password:' field with a masked password '*****', a 'Confirm:' field with a masked password '*****', and an unchecked checkbox for 'Enable root access from remote machines'. The 'Create An Anonymous Account' section includes a question mark icon and a warning: 'This option will create an anonymous account on this server. Please note that this can lead to an insecure system.' At the bottom, there are three buttons: '< Back', 'Next >', and 'Cancel'.

MySQL Server Instance Configuration Wizard

MySQL Server Instance Configuration

Configure the MySQL Server 5.1 server instance.

Please set the security options.

☒ **Modify Security Settings**

 New root password: ***** Enter the root password.

Confirm: ***** Retype the password.

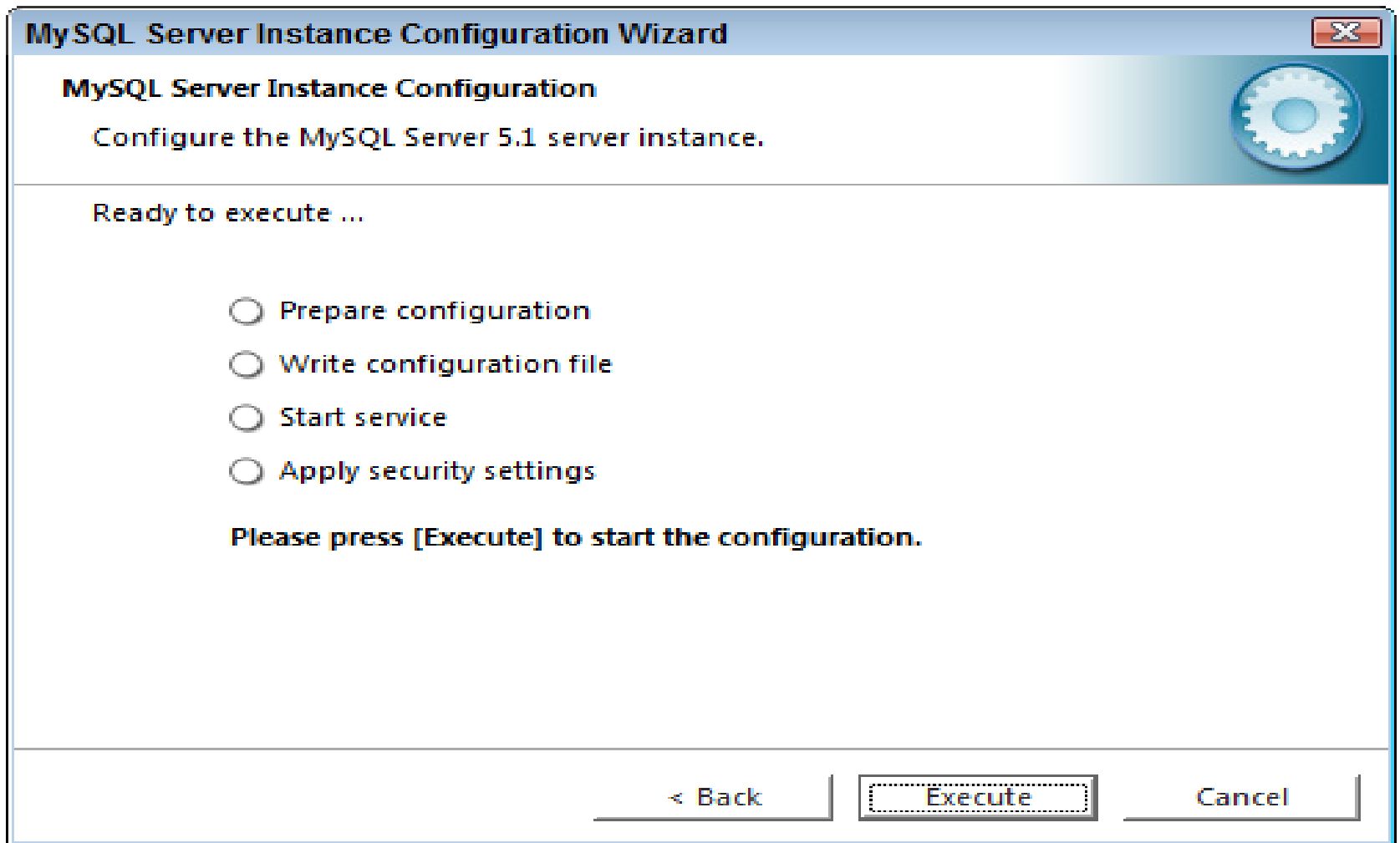
☐ Enable root access from remote machines

☐ **Create An Anonymous Account**

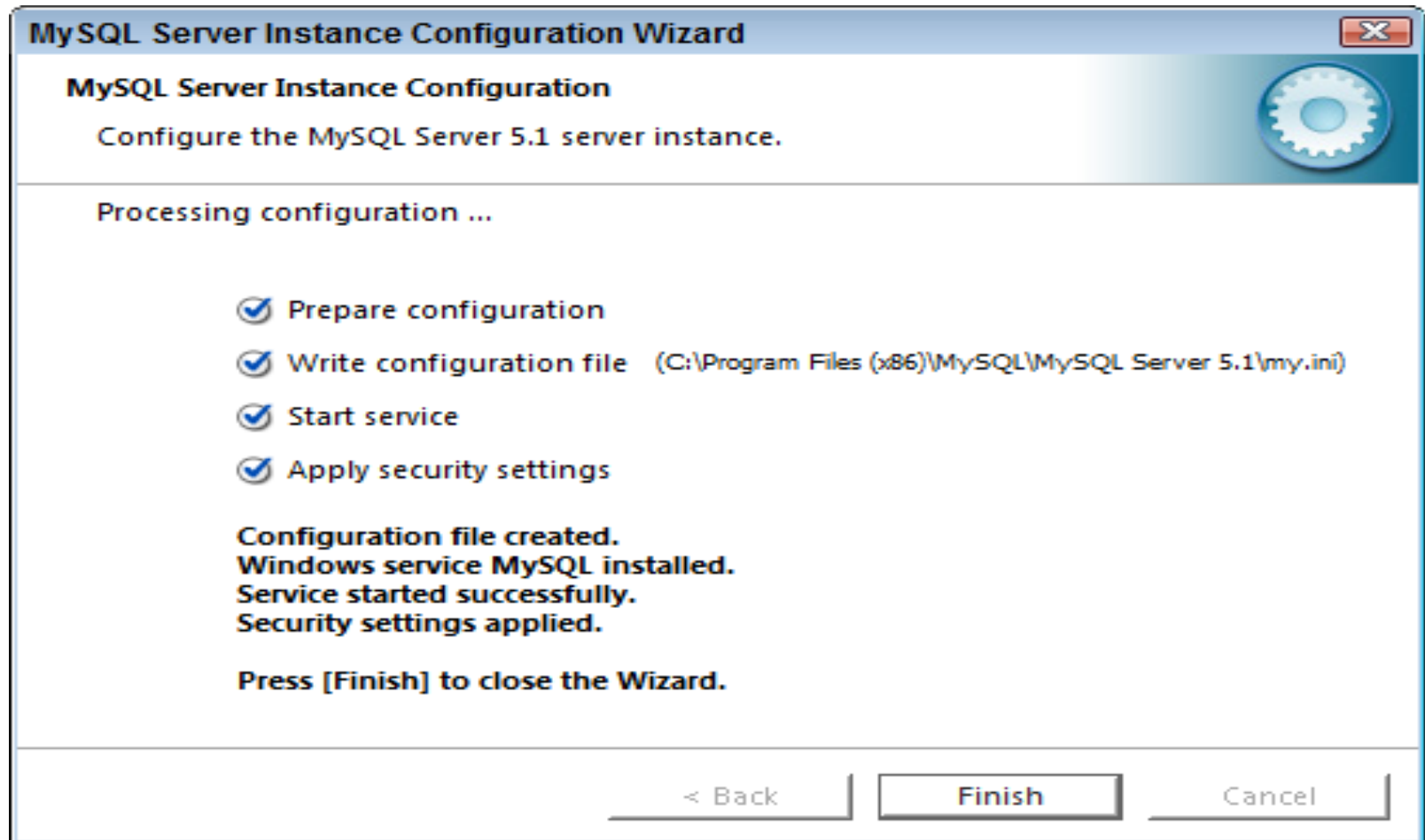
 This option will create an anonymous account on this server. Please note that this can lead to an insecure system.

< Back Next > Cancel

Select the Modify Security Settings option and
select the root password
Click [Next] button



Click [Execute] button



Click [Finish] button

Testing MySQL

Follow the steps to start MySQL

Start → All Programs → MySQL → MySQL Server → MySQL Command Line Client

MySQL will prompt a message to provide password (it requires the same password which was entered during the installation)

C:\Program Files (x86)\MySQL\MySQL Server 4.1\bin\mysql.exe

Enter password: ****

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 3 to server version: 4.1.22-community-nt

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql>

MySQL and SQL

In order to access data within the MySQL database, all programs and user must use SQL(Structured Query Language). SQL is a Language that enables you to create and operate on relational databases.

Classification of SQL statements

- Data Definition Language (DDL) commands
- Data Manipulation Language (DML) commands
- Transaction Control Language (TCL) commands
- Session Control Commands
- System Control Commands

Data Definition Language (DDL) commands

It allows you to perform tasks related to data definition. You can perform tasks like:

➤ **Create, alter and drop schema objects**

This section of DDL commands is used to create or define or change or delete objects such as a table, a view etc. e.g. CREATE TABLE, ALTER TABLE, DROP TABLE etc.

Data Manipulation Language (DML) commands

By data manipulation, we mean :

- the **retrieval** of the information stored in the database.
- the **insertion** of new information into the database
- the **deletion** of the information from the database.
- the **modification** of data stored in the database.

Data Manipulation Language (DML) commands

DML is a language that enables user to access or manipulate data as organized by the appropriate data model. E.g INSERT, UPDATE, DELETE etc.

Transaction Control Language (TCL) commands

These commands manages the changes made by DML commands. E.g.

BEGIN / START TRANSACTION : It marks the beginning of a transaction.

COMMIT: It ends the current transaction by saving database changes and start new transaction.

Transaction Control Language (TCL) commands

ROLLBACK: It ends the current transaction by discarding database changes and start new transaction.

SAVEPOINT: It defines breakpoints for the transaction to allow partial rollbacks.

SIMPLE QUERIES IN SQL

SQL

SQL was developed in 1970s in an IBM Laboratory. SQL sometimes also referred to as SEQUEL is 4th generation non-procedural language.

MySQL SQL Elements

- Literals
- Datatypes
- Nulls
- Comments

Literals

It refer to a fixed data value. This fixed data value may be of character type or numeric literal. E.g. 'Raj' , '8' , '305.8' etc.

Data Types

Data Types are mean to identify the type of data and associated operations of handling it. Data Types in MySQL are divided into three categories:

- Numeric
- Date & Time
- String Type

Numeric Data Type

Common Numeric Data Types are:

- tinyint
- smallint
- mediumint
- int
- bigint

Numeric Data Type

Common Numeric Data Types are:

- `float(m,d)`
- `double(m,d)`
- `decimal(m,d)` or `numeric(m,d)`

Where 'm' is the total no. of digits and 'd' represent no. of decimals.

Date & Time Type

- DATE: stores date in 'YYYY-MM-DD' format.
- DATETIME: stores date & time in 'YYYY-MM-DD HH-MM-SS' format.
- TIMESTAMP: stores date & time in 'YYYYMMDDHHMMSS' format.
- TIME: stores time in 'HH-MM-SS' format.
- YEAR(M): stores a year in 2-digit or 4-digit format.

String/Text Types

- char(m)
- varchar(m)
- blob or text
- tinyblob or tinytext
- mediumblob or mediumtext
- longblob or longtext
- enum

Difference between char & varchar datatypes

char datatype

It specifies a fixed length character string. When a column is given a datatype as char(n), then MySQL ensures that all values stored in that column have this length i.e. n bytes. If a value is shorter than this length n then blanks are added, but the size of variable remains n bytes. Some space is always wasted as all the data elements do not use all the space reserved but processing is much simpler compare to varchar datatype.

Ajmer Singh PGT(CS)

varchar datatype

It specifies a variable length character string. When a column is given a datatype as varchar(n), then the maximum size a value in this column can have n bytes. If a value is shorter than this length n then no blanks are added, but the size of variable remains n bytes. There is no wastage of space but processing is much complex compare to varchar datatype.

Null Value

If a column in a row has no value, then column is said to be null or to contain a NULL.

Comments

A comment is a text that is not executed.
SQL provides following ways to write comment:

- | | |
|------------------------------|------------------|
| ➤ <code>/* comment */</code> | multiline |
| ➤ <code>-- comment</code> | single line |
| ➤ <code># comment</code> | single line |

SQL command syntax

- Keywords
- Commands or statements:- instructions given to sql database.
- clause: consist of command and arguments.

Making Simple Queries

- To understand these consider the following table in the database named “kvr”.

TABLE: EMP

Empid	empname	empphone	empsal	deptno
101	Ajay	234567	20000	25
102	Vijay	654378	15000	30
103	Ramesh	345678	10000	25
104	Ram	346279	15000	10

Accessing Database

To make queries upon the data in tables of a database, you need to open the database for use. SQL provide “use” command for this.

General Syntax:

```
use <databasename>;
```

To view tables in a database

SHOW TABLES;

The SELECT Command

It let you make query on the database. In Simplest form, SELECT statement is used as given below.

```
SELECT <columnname>  
[,<columnname>...] from <tablename>;
```

e.g SELECT empid, empname FROM emp;

SELECTING ALL COLUMNS

General syntax:

```
SELECT * FROM <tablename>;
```

e.g

```
SELECT * FROM emp;
```

REORDERING COLUMNS

Specify the name of column in same order as you want those to get display in SELECT command.

e.g.

e.g `SELECT empname, empid FROM emp;`

Eliminating Redundant Data

The DISTINCT keyword eliminates duplicate rows from the results of a SELECT statement.

e.g.

```
SELECT DISTINCT(deptno) FROM emp;
```

ALL keyword

ALL keyword retains the duplicate rows. It is just the same as when you specify neither DISTINCT nor ALL.

e.g.

```
SELECT ALL(deptno) FROM emp;
```

How to perform Simple Calculation

e.g.

```
SELECT 24*5;
```

DESC or DESCRIBE Command

Description: It is used to view the structure of table.

General Syntax:

DESC <tablename>;

or

DESCRIBE <tablename>;

Example:

DESC myemp;

Scalar Expressions with Selected Field

If you want to perform simple numeric computations on the data to put it in a form more appropriate to your needs, SQL allows you to place scalar expressions and constants among the selected fields.

e.g

```
SELECT empname, empsal+1000 FROM  
emp;
```

Using Column Aliases

It means that column you select in query can be given a different name.

e.g

```
SELECT empname, empsal*0.1 "Bonus"  
FROM emp;
```

Putting text in query Output

e.g

```
SELECT empname, 'gets salary Rs.' empsal  
FROM emp;
```

RELATIONAL OPERATORS

To compare the two values SQL provides the following relational operator:

=

<=

>=

<

>

<>

Relational Operators

a	b	a<b	a<=b	a==b	a>b	a>=b	a!=b
0	0	false	true	true	false	true	false
0	1	true	true	false	false	false	true
1	0	false	false	false	true	true	true
1	1	false	true	true	false	true	false

LOGICAL OPERATORS

To combine the two conditions SQL provides the following logical operator:

OR (||)

AND (&&)

NOT (!)

Logical Operator

LOGICAL OR (||) operator: These return true only if any of the expression is true otherwise false.

LOGICAL AND (&&) operator: These return true only if both of the expression are true otherwise false.

LOGICAL NOT (!) operator: It negates the value of the expression.

SELECTING SPECIFIC ROWS

The WHERE clause in SELECT statement specifies the criteria for the selection of rows to be returned.

General Syntax:

```
SELECT <columnname>  
[,<columnname>...] from <tablename>  
where <condition>;
```

SELECTING SPECIFIC ROWS

e.g.

```
SELECT empname FROM emp WHERE  
empsal < 15000;
```

e.g.

```
SELECT empname FROM emp WHERE  
empid = 101;
```

Condition Based on a Range

The BETWEEN operator defines a range of values that the column values must fall in to make the condition true. The range includes both lower value and upper value.

e.g

```
SELECT empname FROM emp WHERE  
empsal BETWEEN 10000 and 15000;
```

Conditions based on a list

To specify a list of values, IN operator is used.

e.g

```
SELECT empname FROM emp WHERE  
deptno IN ('10','20', '30');
```

Conditions based on Pattern Matches

SQL provides “LIKE” operator for comparison on character strings using patterns. Patterns are described using two special **wildcard** characters.

- percent(%) : The % character matches any substring.
- underscore(_) : The _ matches any character.

Conditions based on Pattern Matches

e.g.

```
SELECT empid FROM emp WHERE  
empname LIKE "A%";
```

e.g

```
SELECT empid FROM emp WHERE  
empname LIKE "_a";
```

Searching for NULL

The NULL value in a column can be searched for in a table using “IS NULL” in the WHERE clause.

```
SELECT empname FROM emp WHERE  
deptno IS NULL;
```

Sorting Results

The ORDER BY clause allows sorting of query results by one or more columns.

e.g.

```
SELECT empname FROM emp ORDER BY  
empsal;
```

SQL FUNCTIONS AND TABLE JOINS

Function

- Function is a special type of predefined commands that performs some operation and return a single value.
- SQL Supports many and many Functions

Function sqrt



e.g
 $x=16$

then $y=4$

Types of SQL Function

- (1) Single Row Function: works with data of single row.
- (2) Multiple Row or Group function: works with data of multiple rows.

Single Row function works with single data like sqrt function whereas multi row function work on more than one value like average function to find average of many numbers.

Multiple Row or Group function

TABLE: EMP

Empid	empname	empphone	empsal	deptno
101	Ajay	234567	20000	25
102	Vijay	654378	15000	30
103	Ramesh	345678	10000	25
104	Ram	346279	15000	10

AVG() function

- This function computes the average of given data.
- Suppose You want to find out the average of salary on EMP table then you can use AVG() function to do that
- e.g.
- SQL> SELECT AVG(empisal) “Average Salary” FROM EMP;

AVG() function

- Output:-
- Average Salary
- -----
- 15000

COUNT() function

- This function computes the number of rows in a given column.
- Suppose You want to find out the number of employees in EMP table then you can use count() function to do that
- e.g.
- SQL> SELECT count(empid) “Total” FROM EMP;

COUNT() function

- Output:-
- Total
- -----
- 5

MAX() function

- This function returns the maximum value for a given column.
- Suppose You want to find out the highest amount of salary paid from EMP table then you can use MAX() function to do that

MAX() function

- Output:-
- Maximum Salary
- -----
- 20000

MIN() function

- This function returns the minimum value for a given column.
- Suppose You want to find out the lowest amount of salary paid from EMP table then you can use MIN() function to do that
- e.g.
- SQL> SELECT MIN(empisal) “Maximum salary” FROM EMP;

MIN() function

- Output:-
- Minimum Salary
- -----
- 10000

SUM() function

- This function computes the sum of values of a given column.
- Suppose You want to find out the total salary paid to employees from EMP table then you can use SUM() function to do that
- e.g.
- SQL> SELECT SUM(empisal) “Total Salary” FROM EMP;

SUM() function

- Output:-
- Total Salary
- -----
- 60000

GROUPING RESULTS

GROUP BY clause

The GROUP BY clause combines all those records that have identical values in a particular column. E.g. To calculate the number of employees in each department. You can use the command.

```
SELECT dept, count(*) FROM emp GROUP  
BY dept;
```

GROUP BY clause

- Output:-

deptno	count(*)
25	2
30	1
10	1

HAVING Clause

The having clause places conditions on groups.

e.g. To calculate the number of employees in department no. 25. You can use the command.

```
SELECT dept, count(*) FROM emp GROUP  
BY dept HAVING dept = 25;
```

HAVING Clause

- Output:-

deptno	count(*)
25	2

Cartesian product

The **Cartesian product**, also referred to as a cross-join, returns all the rows in all the tables listed in the query. Each row in the first table is paired with all the rows in the second table.

The Cartesian Product of Two Relations (cont.)

- Example:

Enrolled

<i>student_id</i>	<i>course_name</i>	<i>credit_status</i>
12345678	CS 105	ugrad
25252525	CS 111	ugrad
45678900	CS 460	grad
33566891	CS 105	non-credit
45678900	CS 510	grad

MajorsIn

<i>student_id</i>	<i>dept_name</i>
12345678	comp sci
45678900	mathematics
25252525	comp sci
45678900	english
66666666	the occult

Enrolled x MajorsIn

<i>Enrolled. student_id</i>	<i>course_name</i>	<i>credit_status</i>	<i>MajorsIn. student_id</i>	<i>dept_name</i>
12345678	CS 105	ugrad	12345678	comp sci
12345678	CS 105	ugrad	45678900	mathematics
12345678	CS 105	ugrad	25252525	comp sci
12345678	CS 105	ugrad	45678900	english
12345678	CS 105	ugrad	66666666	the occult
25252525	CS 111	ugrad	12345678	comp sci
25252525	CS 111	ugrad	45678900	mathematics
...	...	...	...	...



Joins

A join is a query that combines two or more tables.

For example:

```
select * from emp, dept;
```

will give all the possible combination formed of all the rows of both the tables.

Equi Join

The join in which columns are compared for equality is called equi-join.

Non-equi Join

It is a query that specifies some relationship other than equality between columns of tables.

Natural Join

The Join in which only one of the identical column exist is called natural join.

CROSS JOIN

It is a very basic type of join that simply matches each row from one table to every row from another table.

DBMS Concepts

Database

A database may be defined as a collection of interrelated data stored together to serve multiple applications.

DBMS

A DBMS(Data Base Management System) refers to a software that is responsible for storing, maintaining and utilizing databases.

Database System

A database along with a DBMS is referred to as a database system.

Advantages of Database System

- Databases reduces the data redundancy to a large extent.

Data Redundancy

Duplication of data is known as data redundancy.

Office

Shiv

Panipat

Haryana

Office ← ----- Hostel

Shiv

Panipat

Haryana

Hostel

Shiv

Rohtak

Haryana

Advantages of Database System

- Databases can control data inconsistency to a large extent.
- Databases facilitate sharing of data.
- Databases enforce standards.
- Databases can ensure data security.

Data Security

It refers to the protection of data against accidental or intentional disclosure to unauthorized persons, or unauthorized modification or destruction.

Privacy of Data

It refers to the rights of individuals and organizations to determine themselves when, how, and to what extent information about them is to be transmitted to other.

Advantages of Database System

- Integrity can be maintained through databases.

Data Model

It refers to a set of concepts to describe the structure of a database and certain constraints that the database should obey. Different data models are:

- Relational Data Model
- Hierarchical Data Model
- Network Data Model
- Object Oriented Data Model

Relational Data Model

It was propounded by E.F. Codd of the IBM.

In relational data model, the data is organized into tables(i.e. rows and columns). These tables are called relations.

Relational Data Model Terminology

View

It is a table that does not exist in its own right but is instead derived from one or more underlying base tables.

Degree

It is number of attributes in a table or relation.

Cardinality

It is number of tuples/records in a table or relation.

Primary Key

It is a set of one or more attributes that can uniquely identify tuples within the relation.

Candidate Key

All attribute combinations inside a relation that can serve as primary key are candidate keys.

Alternate Key

A candidate key that is not the primary key is called an alternate key.

Foreign Key

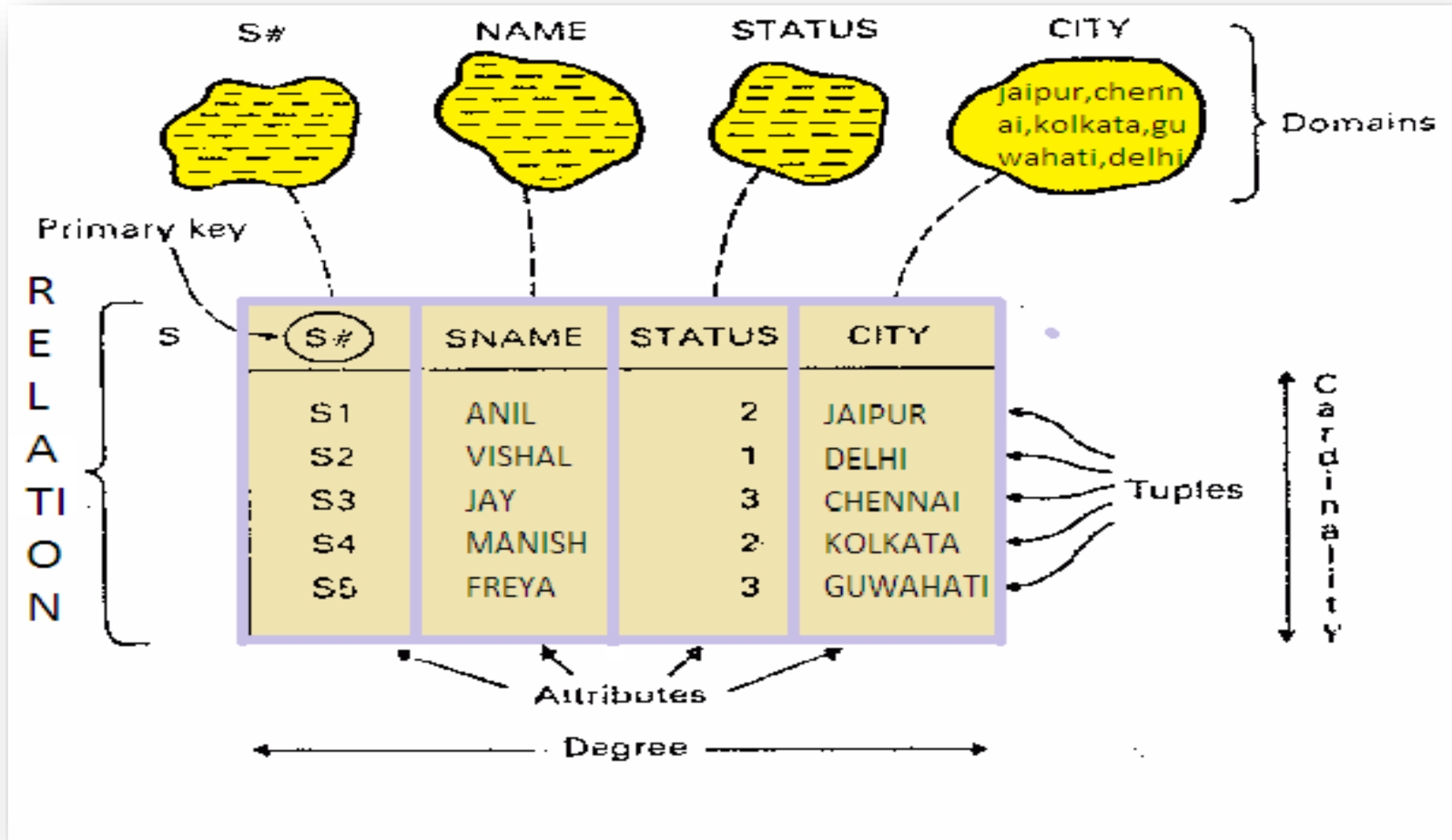
A non-key attribute, whose values are derived from the primary key of some other table, is known as Foreign-Key in the current table.

Referential Integrity

It is a system of rules that DBMS uses to ensure that relationships between records in related tables are valid, and that users don't accidentally delete or change related data.

DATABASE CONCEPTS

RELATIONAL DATABASE TERMS



DATABASE CONCEPTS

KEYS IN A DATABASE

